

PA 1: Implementing xv6 User Commands and System Calls

Task 1. (5 pts) Implement the uptime utility

For this task, I created `uptime.c` in the user directory. The program uses the existing `uptime()` system call to get the number of clock ticks since `xv6` started and then prints the result. I also added the `uptime` program to `UPROGS` in the `Makefile` so that it would compile and run from the `xv6` shell

```
xv6 kernel is booting

init: starting sh
$ uptime
up 47 clock ticks
$
```

Task 2. (10 pts) Implement a `time1` command that reports elapsed time.

Here, I created `time1.c` in the user directory. The program uses `uptime()` to save the starting time, then uses `fork()` to create a child process. The child uses `exec()` to run the command entered by the user, while the parent uses `wait()` to wait for the child to finish. After the command finishes, the program calls `uptime()` again and calculates the elapsed time. I also added `time1` to `UPROGS` in the `Makefile`.

```
xv6 kernel is booting

init: starting sh
$ matmul
Time: 6 ticks
$ time1 matmul
Time: 6 ticks
elapsed time: 6 ticks
$ time1 sleep 10
elapsed time: 10 ticks
$
```

Task 3. (10 pts) Keep track of how much `cputime` a process has used.

For task 3, I changed `kernel/proc.h`, `kernel/proc.c`, and `kernel/trap.c`. In `proc.h`, I added a `cputime` field to `struct proc` so each process can keep track of how much CPU time it uses. In

proc.c, I initialized cputime to 0 when a new process is created in allocproc(). In trap.c, I incremented cputime when a timer interrupt occurs before the process gives up the CPU with yield() in both usertrap() and kerneltrap().

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

Here, I created a new wait2() system call based on the existing wait() system call. I created kernel/pstat.h with a rusage structure that stores the CPU time used by a process. I added the wait2() user declaration and system call entry, assigned it a system call number, and connected it to sys_wait2(). I also created kwait2() in kernel/proc.c. This function waits for a child process to finish like kwait(), but it also copies the child's cputime value back to the parent through the rusage structure.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

For this task, I created time.c, which uses uptime() to measure the elapsed time and the new wait2() system call to get the CPU time used by the child process. The program calculates the CPU percentage by dividing the CPU time by the elapsed time and multiplying by 100. I tested the command using matmul, sleep 10, and two matmul processes running at the same time. The CPU-intensive matmul program used 100% CPU when running by itself, while sleep 10 used 0% CPU. When two matmul programs ran at the same time, they shared the CPU and reported 50% and 58% CPU.

```
-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device v
xv6 kernel is booting
init: starting sh
$ time matmul
Time: 6 ticks
elapsed time: 6 ticks, cpu time: 6 ticks, 100% CPU
$ time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$ time matmul & time matmul &
$ Time: 11 ticks
elapsed time: 12 ticks, cpu time: 6 ticks, 50% CPU
Time: 11 ticks
elapsed time: 12 ticks, cpu time: 7 ticks, 58% CPU
```

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

I learned more about how user commands and system calls work in xv6 in this assignment. I learned how `fork()`, `exec()`, and `wait()` work together to run another program, and how `uptime()` can be used to measure elapsed time. I also learned how to add CPU time tracking to a process and how to create a new `wait2()` system call that sends CPU usage information from the kernel back to a user program. One of the main difficulties I had was getting xv6 to compile correctly with my RISC-V toolchain, so I had to update the Makefile to use the correct 64-bit settings. I also had an issue where the `time` command was not being added to the xv6 filesystem because there was an extra character in the `UPROGS` line. After fixing the Makefile and rebuilding `fs.img`, everything worked correctly. Overall, this assignment helped me understand better how user programs, system calls, processes, and CPU scheduling are connected.

Extra Credit (5 points). Discuss limitations of our `time` command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

One limitation of the `time` command is that it only reports the CPU time used by the direct child process that it waits for. If the program being timed creates its own child processes, the CPU time used by those children would not be included in the result. Another limitation is that the CPU time is measured using clock ticks, so the results are not extremely precise for very short programs. Since the CPU percentage is also calculated using whole-number ticks, the percentage may not always come out to an exact value like 50% even when two processes are sharing the CPU evenly.